

Sorting Algorithms

Now we will understand the complexities of a few simple sorting Algos:

- 1) Insertion sort
- 2) Bubble sort
- 3) Selection sort.

1) Insertion Sort:

Pseudocode:

```
for j = 2 till A.length:
    key = A[j]
    i = j - 1
    while (i > 0 and A[i] > key) {
        A[i+1] = A[i];
        i = i - 1;
    }
    A[i+1] = key;
```

day/date

Dry run of algo:

e.g.

1	2	3	4	5
40	20	60	10	30

remember in algo index starts
from 1.

do iterations one by one.

iteration 1:

$$j = 2$$

$$i = 1$$

$$\text{key} = A[j] = 20$$

now while loop starts:

~~40~~

~~5~~ (a):

$$1 > 0 \quad \&\& \quad 40 > 20$$

so

$$\text{A}[2] = \text{A}[1]$$

40	40	60	10	30
----	----	----	----	----

BHATTI

day/date

(b)

$0 > 0$ ✗ not true
end. while
loop

now do

$A[i+1] = \text{key}$

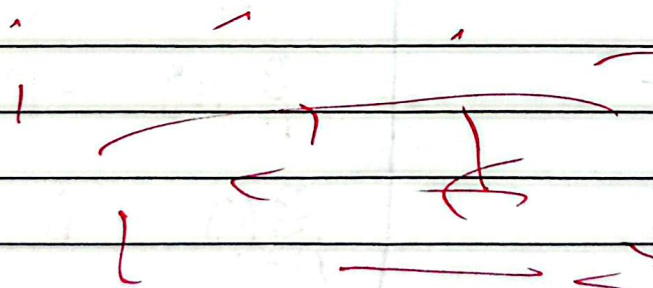
or $i = 0$ to j

$A[1] = 20$

20	40	60	10	50
----	----	----	----	----

2nd iteration

do all
iterations
own self



day/date

Analyzing Algo line by line:

```

for j=2 till A.length: ----- c1
    key = A[j] ----- c2
    i = j-1 ----- c3
    while (i>0 and A[i] > key) { ----- c4
        A[i+1] = A[i]; ----- c5
        i = i-1; ----- c6
    }
    A[j+1] = key ----- c7
  
```

Line	number of times running (work case)	number of times running (Best case)
c1	n	n
c2	$n-1$	$n-1$
c3	$n-1$	$n-1$
c4	$\sum_{j=2}^n t^*j$	$n-1$
c5	$\sum_{j=2}^n (t^*j - 1)$	0
c6	$\sum_{j=2}^n (t^*j - 1)$	0
c7	$n-1$	$n-1$

* t is fast to understand consider $\sum_{j=2}^n j$ only for exp.

Here

$$\sum_{j=1}^n b_j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n (b_j - 1) = \frac{n(n-1)}{2}$$

Best case scenario:

$$F(n) = c_1(n) + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5(0) + c_6(0) + c_7(n-1)$$

$$F(n) = n(c_1 + c_2 + c_3 + c_4 + c_7) - c_4 - c_7$$

Worst case scenario:

$$F(n) = c_1(n) + c_2(n-1) + c_3(n-1) + c_4\left(\frac{n(n+1)}{2} - 1\right) + c_5\left(\frac{n(n-1)}{2}\right) + c_6\left(\frac{n(n-1)}{2}\right) + c_7(n-1)$$

2) Bubble Sort:

Pseudocode:

```

for i = 1 to length[A] ----- c1
do for j = 1 to length[A] down to i+1 --- c2
do if (A[j] < A[j-1]) then --- c3
  exchange (A[j], A[j-1]) --- c4

```

Example dry run:

30	5	20	10
----	---	----	----

iterations -

~~iterations~~

i = 1

j = 4

j = 3

j = 2

i = 2

j = 4

j = 3

day/date

$$i=3$$

$$j=4$$

$$j=4$$

nothing

do we understand that the
inner loop runs
($n-i$) times

Analysis:

	Best case time	Worst case time
C_1	$n+1$	$n+1$
C_2	$\sum_{i=1}^n (n-i+1) = \frac{n(n+1)}{2}$	$\sum_{i=1}^n (n-i+1) = \frac{n(n+1)}{2}$
C_3	$\sum_{i=1}^n (n-i) = \frac{n(n-1)}{2}$	$\sum_{i=1}^n (n-i) = \frac{n(n-1)}{2}$
C_4	0	$\sum_{i=1}^n (n-i) = \frac{n(n-1)}{2}$

Now we find $T(n)$ by multiplying
 $C_1, C_2, \dots, C_4$ to times.

BHATTI

3) Selection sort:

Pseudocode:

•

$n = A.length$ ----- C1

for $i = 1$ To $(n-1)$: ----- C2

$min_idx = i$; ----- C3

 FOR $j = i+1$ to n : ----- C4

 IF $A[j] < A[min_idx]$ ----- C5

$min_idx = j$ ----- C6

 IF $min_idx \neq i$: ----- C7

 exchange ($A[i]$, $A[min_idx]$) ----- C8.

Dry run:

5	2	4	1
---	---	---	---

Outer loop 1 iteration:

$i = 1$

$min_idx = i = 1$

inner loop:

$j = 2$

day/date

is $A[2] < A[1]$

$2 < 4$

✓✓ yes

*

so min-idx ~~2~~ = 2

now $j=3$

$A[3] < A[2]$

$4 < 2$ ✗ no

now $j=4$

$A[4] < A[2]$

$1 < 2$ ✓✓ yes.

min-idx = 4.

so out of outer loop:

enhancing ($A[4], A[1]$)

new array

1	2	4	5
---	---	---	---

BHATTI

day/date

do so on, do all iterations:

Analysis:

code line	Best case	Worst Case
c1	1	1
c2	n	n
c3	n-1	n-1
c4	$\sum_{i=1}^{n-1} (n-i) = \frac{n(n+1)}{2} - 1$	$\frac{n(n+1)-1}{2}$
c5	$\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
c6	0	$\frac{n(n-1)}{2}$
c7	n-1	n-1
c8	0	n-1

BHATTI

day/date

find the worst case & best
case your own self by simply
multiplying. $c_1(\text{cost}) + c_2(\text{cost})$ --